Python Programming - Module 1

Study Material (Syllabus Based)

Module 1.1 - Python Basics

1. Indentation

Python uses indentation (leading whitespace) to define blocks of code instead of braces { }. All statements in the same block must have the same indentation. The standard is 4 spaces. Wrong indentation raises IndentationError.

```
if 5 > 2:
    print("Five is greater")   # inside the if-block
print("Outside the block")
```

2. Comments

- Single-line comment: starts with # and runs to end of line.
- Multi-line comment: Python has no real multi-line comment; triple quotes ''' ... ''' or """ ... """ are used (technically a string / docstring).
- Comments are ignored by the interpreter.

```
# This is a single-line comment
x = 10   # inline comment
'''
This is a
multi-line comment (docstring style)
'''
```

3. Reading Input and Print Output

- input(prompt) reads a line from the keyboard and ALWAYS returns a string (str).
- print() displays output. Parameters: sep (separator, default space), end (default newline), file.
- Use int(), float() to convert input to numbers.

```
name = input("Enter name: ")
age = int(input("Enter age: "))
print("Hello", name, "you are", age)
print("A", "B", "C", sep="-")        # A-B-C
print("Hello", end=" ")
print("World")                       # Hello World  (same line)
```

4. Type Conversions

Converting one data type to another. Two kinds: implicit (done automatically by Python, e.g. int + float -> float) and explicit / type casting (done by programmer using int(), float(), str(), bool(), list(), tuple(), set(), dict()).

```
print(5 + 2.5)         # 7.5  (implicit: int -> float)
print(int("25"))       # 25
print(int(7.9))        # 7  (truncates, no rounding)
print(float(3))        # 3.0
print(str(100) + "1")  # '1001'
print(bool(0), bool("a"))   # False True
print(list("abc"))     # ['a', 'b', 'c']
```

Note: int('3.5') gives ValueError. Use int(float('3.5')). Falsy values: 0, 0.0, '', [], (), {}, set(), None, False.

5. The type() Function and the is Operator

- type(obj) returns the class of the object.

- isinstance(obj, cls) checks if the object is of a given type.
- 'is' compares IDENTITY (same object in memory). '==' compares VALUE. id(obj) gives the memory address.

```
print(type(10))        # <class 'int'>
print(type(3.14))      # <class 'float'>
print(type("hi"))      # <class 'str'>
a = [1, 2]; b = [1, 2]
print(a == b)          # True  (same value)
print(a is b)          # False (different objects)
c = a
print(c is a)          # True
```

6. Dynamically Typed and Strongly Typed

- Dynamically typed: no need to declare the type of a variable. The type is decided at run time from the value and can change when you reassign.
- Strongly typed: Python does not silently convert unrelated types. '5' + 2 raises TypeError; you must convert explicitly.

```
x = 10          # int
x = "ten"       # now str  (dynamic typing)
print("5" + 2)   # TypeError (strong typing)
print("5" + str(2))  # '52'
```

7. Declarations and Assignments

- No declaration keyword. A variable is created the moment a value is assigned using =.
- Multiple assignment: a, b, c = 1, 2, 3
- Same value to many: a = b = c = 0
- Swapping: a, b = b, a
- Augmented assignment: +=, -=, *=, /=, //=, %=, **=

8. Identifiers

An identifier is the name given to a variable, function, class, etc.

- Can contain letters (a-z, A-Z), digits (0-9) and underscore (_).
- Cannot start with a digit.
- Cannot contain spaces or special symbols (@, $, %, -).
- Cannot be a keyword.
- Case-sensitive: Age and age are different.
- No length limit.

Valid: name, _count, total2, myVar. Invalid: 2value, my-var, class, my name.

9. Keywords

Reserved words with a special meaning; cannot be used as identifiers. Python 3 has 35 keywords (count may vary slightly by version). Print them with: import keyword; print(keyword.kwlist)

```
False  None  True  and  as  assert  async  await  break  class  continue
def  del  elif  else  except  finally  for  from  global  if  import  in
is  lambda  nonlocal  not  or  pass  raise  return  try  while  with  yield
```

Note: True, False and None are capitalised. All other keywords are lowercase.

10. Statements and Expressions

- Statement: an instruction that Python executes, e.g. x = 5, print(x), if, for, while, import.
- Expression: a combination of values, variables and operators that evaluates to a single value, e.g. 3 + 4 * 2, x > 5, len(s).
- Every expression can be part of a statement but not every statement is an expression (e.g. assignment).
- A long statement can be continued using backslash \ or inside (), [], {}. Many statements on one line are separated by ;

11. Variables

A variable is a named location that refers to an object in memory. In Python a variable is a reference (label) to a value, not a box that holds the value.

```
x = 5
print(id(x))   # memory address of the object 5
y = x          # both names refer to same object
del x          # deletes the name x
```

12. Operators

Category	Operators	Notes
Arithmetic	+ - * / // % **	/ always gives float; // floor division; ** power
Relational (comparison)	== != > < >= <=	Return True/False; can be chained: 1 < x < 10
Logical	and or not	Short-circuit evaluation
Assignment	= += -= *= /= //= %= **=	x += 1 means x = x + 1
Bitwise	& | ^ ~ << >>	Work on binary bits
Membership	in not in	Check presence in sequence/collection
Identity	is is not	Check same object

```
print(7 / 2)    # 3.5
print(7 // 2)   # 3
print(-7 // 2)  # -4  (floor)
print(7 % 3)    # 1
print(2 ** 3)   # 8
print(5 & 3, 5 | 3, 5 ^ 3)   # 1 7 6
print(3 > 2 and 2 > 1)       # True
```

13. Operator Precedence and Associativity

Precedence decides which operator is evaluated first. Associativity decides the order when operators have the same precedence. Highest to lowest:

Level	Operator	Associativity
1 (highest)	( ) Parentheses	-
2	** Exponent	Right to Left
3	+x -x ~x (unary)	Right to Left
4	* / // %	Left to Right
5	+ -	Left to Right
6	<< >>	Left to Right
7	&	Left to Right
8	^	Left to Right
9	|	Left to Right

Level	Operator	Associativity
10	== != > < >= <= is is not in not in	Left to Right (chained)
11	not	Right to Left
12	and	Left to Right
13 (lowest)	or	Left to Right

```
print(2 + 3 * 4)       # 14
print((2 + 3) * 4)     # 20
print(2 ** 3 ** 2)     # 512  (right to left: 2 ** 9)
print(100 / 10 * 2)    # 20.0 (left to right)
print(not True or True)  # True
```

14. Data Types

Type	Example	Mutable?
int	10, -5, 0b101, 0o17, 0xFF	No
float	3.14, 2e3	No
complex	3 + 4j	No
bool	True, False	No
str	'hello'	No
list	[1, 2, 3]	Yes
tuple	(1, 2, 3)	No
set	{1, 2, 3}	Yes
frozenset	frozenset({1, 2})	No
dict	{'a': 1}	Yes
NoneType	None	No

Numeric: int, float, complex. Sequence: str, list, tuple. Mapping: dict. Set types: set, frozenset. Boolean: bool.

Module 1.2 - Strings and Lists

A. Strings

1. Creating and Storing Strings

A string is an immutable sequence of characters enclosed in single, double or triple quotes.

```
s1 = 'Hello'
s2 = "World"
s3 = '''Multi
line string'''
s4 = "It's fine"
s5 = 'She said "Hi"'
path = r"C:\new\folder"     # raw string, backslash not treated as escape
```

Common escape sequences: \n (newline), \t (tab), \\ (backslash), \' and \" (quotes).

2. Basic String Operations

```
a, b = "Py", "thon"
print(a + b)         # Python    (concatenation)
print(a * 3)         # PyPyPy    (repetition)
print(len("Python")) # 6
print("th" in "Python")      # True  (membership)
print("x" not in "Python")   # True
print("apple" < "banana")    # True  (lexicographic comparison)
```

3. Accessing Characters by Index

- Positive index starts at 0 (left to right); negative index starts at -1 (right to left).
- Out of range index raises IndexError.
- Strings are immutable: s[0] = 'J' raises TypeError.

```
s = "PYTHON"
#    P  Y  T  H  O  N
#    0  1  2  3  4  5
#   -6 -5 -4 -3 -2 -1
print(s[0], s[3], s[-1], s[-3])   # P H N H
```

4. String Slicing and Joining

Syntax: s[start : stop : step]. start is included, stop is excluded. Defaults: start=0, stop=len, step=1.

```
s = "PYTHON"
print(s[1:4])     # YTH
print(s[:3])      # PYT
print(s[3:])      # HON
print(s[::2])     # PTO
print(s[::-1])    # NOHTYP  (reverse)
print(s[-3:])     # HON
print(s[10:])     # ''  (slicing never raises IndexError)
```

Joining: separator.join(iterable_of_strings)

```
words = ["I", "love", "Python"]
print(" ".join(words))     # I love Python
print("-".join("abc"))     # a-b-c
```

5. String Methods

Method	Purpose	Example -> Result
upper() / lower()	Change case	'abc'.upper() -> 'ABC'
title() / capitalize()	Title case / first letter capital	'hi all'.title() -> 'Hi All'
swapcase()	Swap case	'aB'.swapcase() -> 'Ab'
strip() / lstrip() / rstrip()	Remove whitespace/chars	' hi '.strip() -> 'hi'
split(sep)	String to list	'a,b'.split(',') -> ['a','b']
join(iter)	List to string	'-'.join(['a','b']) -> 'a-b'
replace(old, new)	Replace substring	'aab'.replace('a','x') -> 'xxb'
find(sub)	Index of first match, -1 if none	'hello'.find('l') -> 2
index(sub)	Like find but ValueError if none	'hello'.index('l') -> 2
count(sub)	Number of occurrences	'banana'.count('a') -> 3
startswith() / endswith()	Check prefix/suffix	'abc'.startswith('a') -> True
isalpha() / isdigit() / isalnum()	Character class check	'123'.isdigit() -> True
islower() / isupper() / isspace()	Case / space check	'ab'.islower() -> True
center(w) / ljust(w) / rjust(w)	Padding	'hi'.center(6,'*') -> '**hi**'
zfill(w)	Pad with zeros on left	'42'.zfill(5) -> '00042'
partition(sep)	Split into 3-tuple	'a=b'.partition('=') -> ('a','=','b')

Note: String methods never change the original string (immutable); they return a new string.

6. Formatting Strings

```
name, marks = "Ravi", 87.456
# 1) % operator
print("Name: %s, Marks: %.2f" % (name, marks))
# 2) str.format()
print("Name: {}, Marks: {:.1f}".format(name, marks))
print("{1} scored {0}".format(90, "Ravi"))
# 3) f-string (Python 3.6+)
print(f"Name: {name}, Marks: {marks:.2f}")
print(f"{255:b} {255:x} {5:03d} {name:>10}")
```

Format codes: %d / %i integer, %f float, %s string, %c character, %x hex, %o octal. {:.2f} two decimals, {:>10} right-align width 10, {:<10} left-align, {:^10} centre.

B. Lists

1. Creating Lists

A list is an ordered, mutable, heterogeneous collection written in square brackets. It allows duplicates.

```
a = []                    # empty list
b = [1, 2, 3]
c = [1, "two", 3.0, [4, 5]]   # mixed + nested
d = list("abc")           # ['a', 'b', 'c']
e = list(range(1, 6))     # [1, 2, 3, 4, 5]
```

2. Basic List Operations

```
x = [1, 2, 3]; y = [4, 5]
print(x + y)       # [1, 2, 3, 4, 5]
print(x * 2)       # [1, 2, 3, 1, 2, 3]
print(len(x))      # 3
print(2 in x)      # True
for i in x: print(i)
print([1, 2] == [1, 2])   # True
```

3. Indexing and Slicing in Lists

```
L = [10, 20, 30, 40, 50]
print(L[0], L[-1])    # 10 50
print(L[1:4])         # [20, 30, 40]
print(L[::-1])        # [50, 40, 30, 20, 10]
L[1] = 99             # lists are mutable
L[1:3] = [7, 8, 9]    # slice assignment
print(L)              # [10, 7, 8, 9, 40, 50]
```

4. Built-in Functions Used on Lists

Function	Description
len(L)	Number of items
max(L) / min(L)	Largest / smallest item
sum(L)	Sum of numeric items
sorted(L)	Returns NEW sorted list (original unchanged)
reversed(L)	Returns reverse iterator (use list(reversed(L)))
list(x)	Convert iterable to list
any(L) / all(L)	True if any / all items are truthy
enumerate(L)	Pairs of (index, value)

5. List Methods

Method	Description	Example
append(x)	Add x at end	L.append(5)
extend(iter)	Add all items of iterable	L.extend([6,7])
insert(i, x)	Insert x at index i	L.insert(1, 'a')
remove(x)	Remove first x (ValueError if absent)	L.remove(3)
pop([i])	Remove & return item at i (default last)	L.pop()
clear()	Remove all items	L.clear()
index(x)	Index of first x	L.index(30)
count(x)	Occurrences of x	L.count(2)
sort(reverse=False)	Sort IN PLACE, returns None	L.sort(reverse=True)
reverse()	Reverse IN PLACE	L.reverse()
copy()	Shallow copy	M = L.copy()

Note: append() adds ONE element (a list added by append stays nested); extend() adds each element separately. sort() returns None, so L = L.sort() is a common mistake.

```
L = [1, 2]
L.append([3, 4]);  print(L)   # [1, 2, [3, 4]]
M = [1, 2]
M.extend([3, 4]);  print(M)   # [1, 2, 3, 4]
```

6. The del Statement

```
L = [10, 20, 30, 40, 50]
del L[1]        # [10, 30, 40, 50]
del L[1:3]      # [10, 50]
del L           # deletes the whole list variable
```

del removes by index/slice (and no value is returned), remove() removes by value, pop() removes by index and returns the value.

Module 1.3 - Dictionaries

1. Creating a Dictionary

A dictionary is an unordered-by-position (insertion-ordered since Python 3.7), mutable collection of key : value pairs in curly braces. Keys must be unique and immutable (int, str, tuple); values can be anything and can repeat.

```
d1 = {}                                 # empty dict
d2 = {"name": "Asha", "age": 20}
d3 = dict(name="Asha", age=20)
d4 = dict([("a", 1), ("b", 2)])
d5 = dict(zip(["x", "y"], [10, 20]))    # {'x': 10, 'y': 20}
```

Note: {} creates an empty DICT, not an empty set. Use set() for an empty set.

2. Accessing and Modifying key : value Pairs

```
d = {"name": "Asha", "age": 20}
print(d["name"])          # Asha
print(d.get("city"))      # None   (no error)
print(d.get("city", "NA"))# NA     (default)
# print(d["city"])        # KeyError
d["age"] = 21             # modify
d["city"] = "Pune"        # add new pair
print("age" in d)         # True   (membership checks KEYS)
for k in d: print(k, d[k])
for k, v in d.items(): print(k, v)
```

3. Built-in Functions Used on Dictionaries

Function	Description
len(d)	Number of key-value pairs
sorted(d)	Sorted list of keys
min(d) / max(d)	Smallest / largest key
sum(d)	Sum of keys (if numeric)
any(d) / all(d)	Truthiness test on keys
dict()	Create a dictionary
list(d) / tuple(d) / set(d)	Convert keys into that type
type(d)	<class 'dict'>

4. Dictionary Methods

Method	Description
keys()	View of all keys
values()	View of all values
items()	View of (key, value) tuples
get(k, default)	Value for k, else default (None)
update(other)	Add/overwrite pairs from another dict
pop(k)	Remove key k and return its value (KeyError if absent)
popitem()	Remove and return the last inserted pair
setdefault(k, v)	Return d[k]; if missing, insert k:v and return v
clear()	Remove all pairs
copy()	Shallow copy
fromkeys(seq, v)	New dict with keys from seq, all set to v

```
d = {"a": 1, "b": 2}
d.update({"b": 20, "c": 3})
print(d)                        # {'a': 1, 'b': 20, 'c': 3}
print(d.pop("a"))               # 1
print(list(d.keys()))           # ['b', 'c']
print(dict.fromkeys(["x", "y"], 0))   # {'x': 0, 'y': 0}
```

5. The del Statement

```
d = {"a": 1, "b": 2, "c": 3}
del d["b"]       # {'a': 1, 'c': 3}
del d            # deletes entire dictionary
```

Common program: word / character frequency

```
text = "banana"
freq = {}
for ch in text:
    freq[ch] = freq.get(ch, 0) + 1
print(freq)      # {'b': 1, 'a': 3, 'n': 2}
```

Module 1.4 - Tuples and Sets

A. Tuples

1. Creating Tuples

A tuple is an ordered, IMMUTABLE collection written in parentheses. It allows duplicates and mixed types.

```
t1 = ()                 # empty tuple
t2 = (1, 2, 3)
t3 = 1, 2, 3            # packing, parentheses optional
t4 = (5,)               # single-element tuple needs a comma
t5 = tuple([1, 2, 3])
print(type((5)), type((5,)))   # <class 'int'> <class 'tuple'>
```

2. Basic Tuple Operations

```
a = (1, 2); b = (3, 4)
print(a + b)       # (1, 2, 3, 4)
print(a * 2)       # (1, 2, 1, 2)
print(len(a), 2 in a)
x, y = a           # unpacking
print(x, y)        # 1 2
```

3. Indexing and Slicing in Tuples

```
t = (10, 20, 30, 40, 50)
print(t[0], t[-1])    # 10 50
print(t[1:4])         # (20, 30, 40)
print(t[::-1])        # (50, 40, 30, 20, 10)
# t[0] = 5            # TypeError: tuple is immutable
```

4. Built-in Functions Used on Tuples

len(), max(), min(), sum(), sorted() (returns a LIST), tuple(), any(), all(), enumerate(), reversed().

5. Tuple Methods

Only two methods because tuples are immutable: count(x) and index(x).

```
t = (1, 2, 2, 3)
print(t.count(2))   # 2
print(t.index(3))   # 3
```

6. Relation between Tuples and Lists

Feature	List	Tuple
Brackets	[ ]	( )
Mutable	Yes	No
Methods	Many (append, sort ...)	Only count, index
Speed / memory	Slower, more memory	Faster, less memory
Use as dict key	No	Yes (if all items hashable)
Use	Data that changes	Fixed data, records

```
L = list((1, 2, 3))      # tuple -> list  [1, 2, 3]
T = tuple([1, 2, 3])     # list -> tuple  (1, 2, 3)
# To 'modify' a tuple: convert to list, change, convert back
```

Note: A tuple containing a list is not fully frozen: t = (1, [2, 3]); t[1].append(4) works.

7. Relation between Tuples and Dictionaries

- d.items() returns (key, value) pairs as tuples.
- A list of 2-item tuples can be converted to a dictionary using dict().
- Tuples can be used as dictionary keys (lists cannot).

```
d = {"a": 1, "b": 2}
print(list(d.items()))             # [('a', 1), ('b', 2)]
print(dict([("x", 10), ("y", 20)]))# {'x': 10, 'y': 20}
loc = {(19.07, 72.87): "Mumbai"}   # tuple as key
```

8. The zip() Function

zip(iter1, iter2, ...) pairs up elements from iterables into tuples. It stops at the shortest iterable and returns an iterator (wrap with list()/dict()).

```
names = ["A", "B", "C"]
marks = [90, 80, 70]
print(list(zip(names, marks)))   # [('A', 90), ('B', 80), ('C', 70)]
print(dict(zip(names, marks)))   # {'A': 90, 'B': 80, 'C': 70}
for n, m in zip(names, marks):
    print(n, m)
a, b = zip(*[(1, 'x'), (2, 'y')])   # unzip -> (1, 2) ('x', 'y')
```

B. Sets

1. Creating Sets

A set is an UNORDERED collection of UNIQUE, immutable elements written in curly braces. Sets are mutable (can add/remove) but cannot be indexed or sliced. Duplicates are removed automatically.

```
s1 = {1, 2, 3, 2, 1}        # {1, 2, 3}
s2 = set([1, 2, 2, 3])      # {1, 2, 3}
s3 = set("hello")           # {'h', 'e', 'l', 'o'}  (order may vary)
empty = set()               # NOT {}
# s = {1, [2, 3]}           # TypeError: list is unhashable
```

2. Set Methods

Method	Description
add(x)	Add one element
update(iter)	Add many elements
remove(x)	Remove x; KeyError if absent
discard(x)	Remove x; no error if absent
pop()	Remove and return an arbitrary element
clear()	Remove all elements
copy()	Shallow copy
union(s) or |	All elements of both
intersection(s) or &	Common elements
difference(s) or -	In first but not in second
symmetric_difference(s) or ^	In either but not both
issubset(s) / issuperset(s)	Subset / superset test
isdisjoint(s)	True if no common elements

```
A = {1, 2, 3, 4}; B = {3, 4, 5}
print(A | B)    # {1, 2, 3, 4, 5}
print(A & B)    # {3, 4}
print(A - B)    # {1, 2}
print(A ^ B)    # {1, 2, 5}
print({1, 2}.issubset(A))   # True
```

3. Traversing of Sets

```
s = {10, 20, 30}
for item in s:
    print(item)           # order is not guaranteed
for i, item in enumerate(sorted(s)):
    print(i, item)        # sorted order
```

4. frozenset()

frozenset is an immutable set. It supports all read-only set operations (union, intersection, difference, issubset...) but NOT add/remove/update. Because it is hashable it can be used as a dictionary key or as an element inside another set.

```
fs = frozenset([1, 2, 3, 2])
print(fs)               # frozenset({1, 2, 3})
print(fs | {4})         # frozenset({1, 2, 3, 4})
# fs.add(5)             # AttributeError
d = {frozenset([1, 2]): "pair"}
```

Quick Revision - Collections at a Glance

	String	List	Tuple	Set	Dict
Syntax	'abc'	[1,2]	(1,2)	{1,2}	{k:v}
Ordered	Yes	Yes	Yes	No	Insertion order
Mutable	No	Yes	No	Yes	Yes
Duplicates	Yes	Yes	Yes	No	Keys: No
Indexing	Yes	Yes	Yes	No	By key
Empty	''	[]	()	set()	{}

Common Mistakes to Avoid in the Exam

- input() returns a string - convert with int()/float() before arithmetic.
- (5) is an int; (5,) is a tuple.
- {} is an empty dictionary, set() is an empty set.
- list.sort() returns None. Use sorted() if you need the result.
- Slicing stop index is excluded; slicing never raises IndexError but indexing does.
- Strings and tuples are immutable - item assignment gives TypeError.
- remove() on a missing list item gives ValueError; on a missing set item gives KeyError; use discard().
- dict[key] on a missing key gives KeyError; use get().
- Use == for value comparison and is for identity comparison.